

**Nome:** Guilherme de Moraes China Costa

**Data:** 14/02/2026

**Título:** Engenharia de Software: Refatoração - TP1

**Link da implementação e evidências:**

<https://github.com/guilhermecosta/INFNET/tree/master/refactoring/TP1>

As imagens podem não estar muito boas aqui. Melhor ver no repo, juntamente com o código.

```
12 package com.gildedrose.updater;
13
14 import com.gildedrose.Item;
15
16 /**
17  * Atualizador para itens "Conjured".
18  * Itens Conjured diminuem a qualidade duas vezes mais rápido que os itens normais.
19  */
20 public class ConjuredItemUpdater implements ItemUpdater {
21
22     private static final String CONJURED = "Conjured";
23     private static final int MIN_QUALITY = 0;
24     private static final int DOUBLE_DECAY_RATE = 2;
25
26     @Override
27     public void update(Item item) {
28         if (item.quality > MIN_QUALITY) {
29             item.quality = item.quality - DOUBLE_DECAY_RATE;
30             if (item.quality < MIN_QUALITY) {
31                 item.quality = MIN_QUALITY;
32             }
33         }
34
35         item.sellIn = item.sellIn - 1;
36
37         if (item.sellIn < 0) {
38             if (item.quality > MIN_QUALITY) {
39                 item.quality = item.quality - DOUBLE_DECAY_RATE;
40                 if (item.quality < MIN_QUALITY) {
41                     item.quality = MIN_QUALITY;
42                 }
43             }
44         }
45     }
46
47     @Override
48     public boolean canUpdate(Item item) {
49         return CONJURED.equals(item.name);
50     }
51 }
```

```
41 import com.gildedrose.updater.BackstagePassUpdater;
42 import com.gildedrose.updater.DefaultUpdater;
43 import com.gildedrose.updater.ConjuredItemUpdater;
44
45 /**
46  * Sistema de controle de estoque da loja Gilded Rose.
47  * Refatorado usando o padrão Strategy para permitir fácil adição de novos tipos de itens.
48  *
49  * A estrutura atual facilita a adição de novos tipos de item? Justifique com base no Princípio Aberto-Fechado:
50  * Sim. O Princípio Aberto-Fechado (OCP) estabelece que entidades de software devem ser abertas para extensão mas fechadas para modificação.
51  * A implementação atual usa o padrão Strategy com a interface ItemUpdater. Para adicionar um novo tipo de item,
52  * basta criar uma nova classe que implemente ItemUpdater e adicioná-la à lista de updaters, sem alterar
53  * a classe GildedRose ou as classes updaters existentes.
54  *
55  * A implementação dos ItemUpdater respeita o Princípio da Responsabilidade Única? Explique:
56  * Sim. Cada classe ItemUpdater tem uma única responsabilidade: atualizar um tipo específico de item.
57  * AgedBrieUpdater só sabe como atualizar Aged Brie, BackstagePassUpdater só atualiza passes, etc.
58  * Isso segue o SRP, pois cada classe tem apenas uma razão para mudar: uma mudança nas regras daquele tipo de item.
59  *
60  * Alguma violação do Princípio de Substituição de Liskov pode ser identificada em sua hierarquia? Se sim, corrija-a:
61  * O DefaultUpdater retorna true para canUpdate() para qualquer item, o que poderia ser um problema.
62  * No entanto, como ele é usado como último recurso na lista de updaters (Strategy pattern), isso funciona corretamente.
63  * Uma alternativa seria usar null ou Optional para representar "nenhum updater encontrado", mas a abordagem atual
64  * é um padrão comum em implementações de Strategy onde há um caso default.
65  */
66
67 class GildedRose {
68     private final Item[] items;
69     private final ItemUpdater[] updaters;
70
71     public GildedRose(Item[] items) {
72         this.items = items;
73     }
74
75     public void updateQuality() {
76         for (int i = 0; i < items.length; i++) {
77             Item item = items[i];
78             if (item != null) {
79                 for (ItemUpdater updater : updaters) {
80                     if (updater.canUpdate(item)) {
81                         updater.update(item);
82                     }
83                 }
84             }
85         }
86     }
87 }
```

